Preparación de datos demográficos de OWID (2000–2022)

Objetivo: descargar cuatro conjuntos de datos, revisar sus columnas, preparar cada uno por separado y unirlos por `Entity` y `Year`. El resultado servirá para una práctica posterior de aprendizaje automático.

Ejecuta las celdas en orden. Requiere `pandas` y `matplotlib`, además de acceso a internet. Fuente: Our World in Data.

In [3]:
```
import pandas as pd
import matplotlib.pyplot as plt

OPCIONES_OWID = {"User-Agent": "Our World In Data data fetch/1.0"}
BASE = "https://ourworldindata.org/grapher/"
SUFIJO = ".csv?v=1&csvType=full&useColumnShortNames=true"
```

1. Descargar los cuatro archivos

Las direcciones son cadenas de texto simples. Cada archivo se carga en su propio DataFrame.

In [4]:
```
url_population = BASE + "population-with-un-projections" + SUFIJO
url_births = BASE + "number-of-births-per-year" + SUFIJO
url_deaths = BASE + "number-of-deaths-per-year" + SUFIJO
url_child_deaths = BASE + "number-of-child-deaths-unwpp" + SUFIJO

population = pd.read_csv(url_population, storage_options=OPCIONES_OWID)
births = pd.read_csv(url_births, storage_options=OPCIONES_OWID)
deaths = pd.read_csv(url_deaths, storage_options=OPCIONES_OWID)
child_deaths = pd.read_csv(url_child_deaths, storage_options=OPCIONES_OWID)

print("Filas y columnas:")
print("population:", population.shape)
print("births:", births.shape)
print("deaths:", deaths.shape)
print("child_deaths:", child_deaths.shape)
```

```
Filas y columnas:
population: (39562, 5)
births: (39109, 5)
deaths: (38656, 5)
child_deaths: (18942, 4)
```

2. Explorar las columnas

OWID puede modificar los nombres de los indicadores. Revisamos las columnas reales antes de seleccionarlas.

In [5]:
```
for nombre, tabla in [("population", population),
                      ("births", births),
                      ("deaths", deaths),
                      ("child_deaths", child_deaths)]:
    print(f"\n{nombre}: {tabla.columns.tolist()}")
    display(tabla.head(3))
```

```
population: ['entity', 'code', 'year', 'population__sex_all__age_all__variant_estima
tes', 'population__sex_all__age_all__variant_medium__projected']
```

	entity	code	year	population__sex_all__age_all__variant_estimates	population__sex_all
0	Afghanistan	AFG	1950	7776180.0	
1	Afghanistan	AFG	1951	7879343.0	
2	Afghanistan	AFG	1952	7987784.0	

```
births: ['entity', 'code', 'year', 'births__sex_all__age_all__variant_estimates', 'b
irths__sex_all__age_all__variant_medium__projected']
```

	entity	code	year	births__sex_all__age_all__variant_estimates	births__sex_all__age_all__
0	Afghanistan	AFG	1950	383985.0	
1	Afghanistan	AFG	1951	391002.0	
2	Afghanistan	AFG	1952	397663.0	

```
deaths: ['entity', 'code', 'year', 'deaths__sex_all__age_all__variant_estimates', 'd
eaths__sex_all__age_all__variant_medium__projected']
```

	entity	code	year	deaths__sex_all__age_all__variant_estimates	deaths__sex_all__age_all
0	Afghanistan	AFG	1950	290972.0	
1	Afghanistan	AFG	1951	288752.0	
2	Afghanistan	AFG	1952	288059.0	

```
child_deaths: ['entity', 'code', 'year', 'deaths__sex_all__age_0_4__variant_estimate
s']
```

	entity	code	year	deaths__sex_all__age_0_4__variant_estimates
0	Afghanistan	AFG	1950	160791
1	Afghanistan	AFG	1951	158566
2	Afghanistan	AFG	1952	157832

3. Preparar población

Elegimos la columna de **estimaciones históricas** para ambos sexos y todas las edades; excluimos las proyecciones. Después conservamos los años de interés.

In [6]:
```
columnas_population = [
    columna for columna in population.columns
        if "population" in columna and "estimates" in columna]

print("Columnas candidatas:", columnas_population)

assert len(columnas_population) == 1, "Revisar manualmente el indicador de població


col_population = columnas_population[0]

population_limpio = population[["entity", "year", col_population]].copy()

population_limpio = population_limpio.rename(columns={col_population: "population"}

population_limpio = population_limpio[population_limpio["year"].between(2000, 2022)

print(population_limpio.shape)
display(population_limpio.head())
```

```
Columnas candidatas: ['population__sex_all__age_all__variant_estimates']
(6026, 3)
```

	entity	year	population
50	Afghanistan	2000	20130334.0
51	Afghanistan	2001	20284303.0
52	Afghanistan	2002	21378123.0
53	Afghanistan	2003	22733053.0
54	Afghanistan	2004	23560656.0

4. Preparar nacimientos

Seleccionamos el número anual de nacimientos históricos. Revisamos qué entidades comparte con las otras tablas.

In [7]:
```
columnas_births = [c for c in births.columns
                   if "births" in c and "estimates" in c]


print("Columnas candidatas:", columnas_births)
assert len(columnas_births) == 1, "Revisar manualmente el indicador de nacimientos
col_births = columnas_births[0]
```

```
births_limpio = births[["entity", "year", col_births]].copy()
births_limpio = births_limpio.rename(columns={col_births: "births"})
births_limpio = births_limpio[births_limpio["year"].between(2000, 2022)].copy()
print(births_limpio.shape)
display(births_limpio.head())
```

```
Columnas candidatas: ['births__sex_all__age_all__variant_estimates']
(5957, 3)
```

	entity	year	births
50	Afghanistan	2000	1036525.0
51	Afghanistan	2001	1006562.0
52	Afghanistan	2002	1015202.0
53	Afghanistan	2003	1069727.0
54	Afghanistan	2004	1095372.0

5. Preparar defunciones totales

In [8]:
```
columnas_deaths = [c for c in deaths.columns
                   if "deaths" in c and "estimates" in c]

print("Columnas candidatas:", columnas_deaths)
assert len(columnas_deaths) == 1, "Revisar manualmente el indicador de defunciones

col_deaths = columnas_deaths[0]

deaths_limpio = deaths[["entity", "year", col_deaths]].copy()
deaths_limpio = deaths_limpio.rename(columns={col_deaths: "deaths"})
deaths_limpio = deaths_limpio[deaths_limpio["year"].between(2000, 2022)].copy()

print(deaths_limpio.shape)
display(deaths_limpio.head())
```

```
Columnas candidatas: ['deaths__sex_all__age_all__variant_estimates']
(5888, 3)
```

	entity	year	deaths
50	Afghanistan	2000	253804.0
51	Afghanistan	2001	242473.0
52	Afghanistan	2002	239161.0
53	Afghanistan	2003	243573.0
54	Afghanistan	2004	244435.0

6. Preparar defunciones de menores de cinco años

Este indicador corresponde a niños de 0 a 4 años; no debe confundirse con mortalidad de menores de un año.

In [9]:
```
columnas_child = [c for c in child_deaths.columns
                  if c not in ["entity", "code", "year"]]

print("Columnas candidatas:", columnas_child)

assert len(columnas_child) == 1, "Revisar manualmente el indicador de menores de ci

col_child = columnas_child[0]

child_deaths_limpio = child_deaths[["entity", "year", col_child]].copy()
child_deaths_limpio = child_deaths_limpio.rename(columns={col_child: "child_deaths_

child_deaths_limpio = child_deaths_limpio[child_deaths_limpio["year"].between(2000,

print(child_deaths_limpio.shape)
display(child_deaths_limpio.head())
```

```
Columnas candidatas: ['deaths__sex_all__age_0_4__variant_estimates']
(5887, 3)
```

	entity	year	child_deaths_under_5
50	Afghanistan	2000	133112
51	Afghanistan	2001	125627
52	Afghanistan	2002	123215
53	Afghanistan	2003	123894
54	Afghanistan	2004	122315

7. Revisar llaves y cobertura

Cada combinación `Entity` – `Year` debe aparecer una sola vez en cada tabla. `Code` no se utiliza como llave porque puede faltar en agregados regionales.

In [10]:
```
tablas = [("population", population_limpio), ("births", births_limpio),
          ("deaths", deaths_limpio), ("child_deaths", child_deaths_limpio)]

for nombre, tabla in tablas:
    duplicados = tabla.duplicated(["entity", "year"]).sum()
    print(nombre, "| entidades:", tabla["entity"].nunique(),
          "| años:", tabla["year"].min(), "a", tabla["year"].max(),
          "| llaves duplicadas:", duplicados)
    assert duplicados == 0, f"Llaves duplicadas en {nombre}"

entidades_comunes = set(population_limpio["entity"]) & set(births_limpio["entity"])
                  & set(deaths_limpio["entity"]) & set(child_deaths_limpio["entity"
```

```
print("Entidades presentes en los cuatro archivos:", len(entidades_comunes))
print(sorted(entidades_comunes)[:30])
```

```
population | entidades: 262 | años: 2000 a 2022 | llaves duplicadas: 0
births | entidades: 259 | años: 2000 a 2022 | llaves duplicadas: 0
deaths | entidades: 256 | años: 2000 a 2022 | llaves duplicadas: 0
child_deaths | entidades: 256 | años: 2000 a 2022 | llaves duplicadas: 0
Entidades presentes en los cuatro archivos: 253
['Afghanistan', 'Africa (UN)', 'Albania', 'Algeria', 'American Samoa', 'Andorra', 'A
ngola', 'Anguilla', 'Antigua and Barbuda', 'Argentina', 'Armenia', 'Aruba', 'Asia (U
N)', 'Australia', 'Austria', 'Azerbaijan', 'Bahamas', 'Bahrain', 'Bangladesh', 'Barb
ados', 'Belarus', 'Belgium', 'Belize', 'Benin', 'Bermuda', 'Bhutan', 'Bolivia', 'Bon
aire Sint Eustatius and Saba', 'Bosnia and Herzegovina', 'Botswana']
```

8. Unir las tablas, un paso a la vez

Usamos `inner` : permanecen únicamente las combinaciones de entidad y año presentes en ambos DataFrames. `validate="one_to_one"` detecta llaves duplicadas.

In [11]:
```
population_births = pd.merge(
    population_limpio, births_limpio,
    on=["entity", "year"], how="inner", validate="one_to_one"
)
print("Después de población + nacimientos:", population_births.shape)
display(population_births.head())
```

```
Después de población + nacimientos: (5957, 4)
```

	entity	year	population	births
0	Afghanistan	2000	20130334.0	1036525.0
1	Afghanistan	2001	20284303.0	1006562.0
2	Afghanistan	2002	21378123.0	1015202.0
3	Afghanistan	2003	22733053.0	1069727.0
4	Afghanistan	2004	23560656.0	1095372.0

In [12]:
```
population_births_deaths = pd.merge(
    population_births, deaths_limpio,
    on=["entity", "year"], how="inner", validate="one_to_one"
)
print("Después de agregar defunciones:", population_births_deaths.shape)
display(population_births_deaths.head())
```

```
Después de agregar defunciones: (5819, 5)
```

	entity	year	population	births	deaths
0	Afghanistan	2000	20130334.0	1036525.0	253804.0
1	Afghanistan	2001	20284303.0	1006562.0	242473.0
2	Afghanistan	2002	21378123.0	1015202.0	239161.0
3	Afghanistan	2003	22733053.0	1069727.0	243573.0
4	Afghanistan	2004	23560656.0	1095372.0	244435.0

In [13]:
```
demografia = pd.merge(
    population_births_deaths, child_deaths_limpio,
    on=["entity", "year"], how="inner", validate="one_to_one"
)
print("Dataset unido:", demografia.shape)
display(demografia.head())
```

```
Dataset unido: (5818, 6)
```

	entity	year	population	births	deaths	child_deaths_under_5
0	Afghanistan	2000	20130334.0	1036525.0	253804.0	133112
1	Afghanistan	2001	20284303.0	1006562.0	242473.0	125627
2	Afghanistan	2002	21378123.0	1015202.0	239161.0	123215
3	Afghanistan	2003	22733053.0	1069727.0	243573.0	123894
4	Afghanistan	2004	23560656.0	1095372.0	244435.0	122315

9. Revisar faltantes y crear tasas

Eliminamos solo filas con algún indicador necesario ausente. Las tasas de nacimientos y defunciones se expresan por 1,000 habitantes. La razón de muertes de menores de cinco años usa nacimientos del **mismo año** como denominador; es una medida descriptiva aproximada, no una tasa de mortalidad de cohorte.

In [14]:
```
print("Valores faltantes antes de limpiar:")
print(demografia.isna().sum())
print("Filas antes de limpiar:", len(demografia))

demografia = demografia.dropna(subset=["population", "births", "deaths", "child_dea
demografia = demografia[(demografia["population"] > 0) & (demografia["births"] > 0)

print("Filas después de limpiar:", len(demografia))
print("Entidades después de limpiar:", demografia["entity"].nunique())
assert not demografia.empty, "No hay entidades y años coincidentes; revisar fuentes
```

```
Valores faltantes antes de limpiar:
entity                  0
year                    0
population              0
births                  0
deaths                  0
child_deaths_under_5    0
dtype: int64
Filas antes de limpiar: 5818
Filas después de limpiar: 5818
Entidades después de limpiar: 253
```

In [15]:
```
demografia["birth_rate_per_1000"] = demografia["births"] / demografia["population"]
demografia["death_rate_per_1000"] = demografia["deaths"] / demografia["population"]
demografia["under5_deaths_per_1000_births_same_year"] = (
    demografia["child_deaths_under_5"] / demografia["births"] * 1000
)

demografia = demografia[[
    "entity", "year", "population", "births", "deaths", "child_deaths_under_5",
    "birth_rate_per_1000", "death_rate_per_1000",
    "under5_deaths_per_1000_births_same_year"
]].sort_values(["entity", "year"]).reset_index(drop=True)

display(demografia.head())
display(demografia.describe(include="all"))
```

	entity	year	population	births	deaths	child_deaths_under_5	birth_rate_per_1(
0	Afghanistan	2000	20130334.0	1036525.0	253804.0	133112	51.490
1	Afghanistan	2001	20284303.0	1006562.0	242473.0	125627	49.622
2	Afghanistan	2002	21378123.0	1015202.0	239161.0	123215	47.487
3	Afghanistan	2003	22733053.0	1069727.0	243573.0	123894	47.056
4	Afghanistan	2004	23560656.0	1095372.0	244435.0	122315	46.491

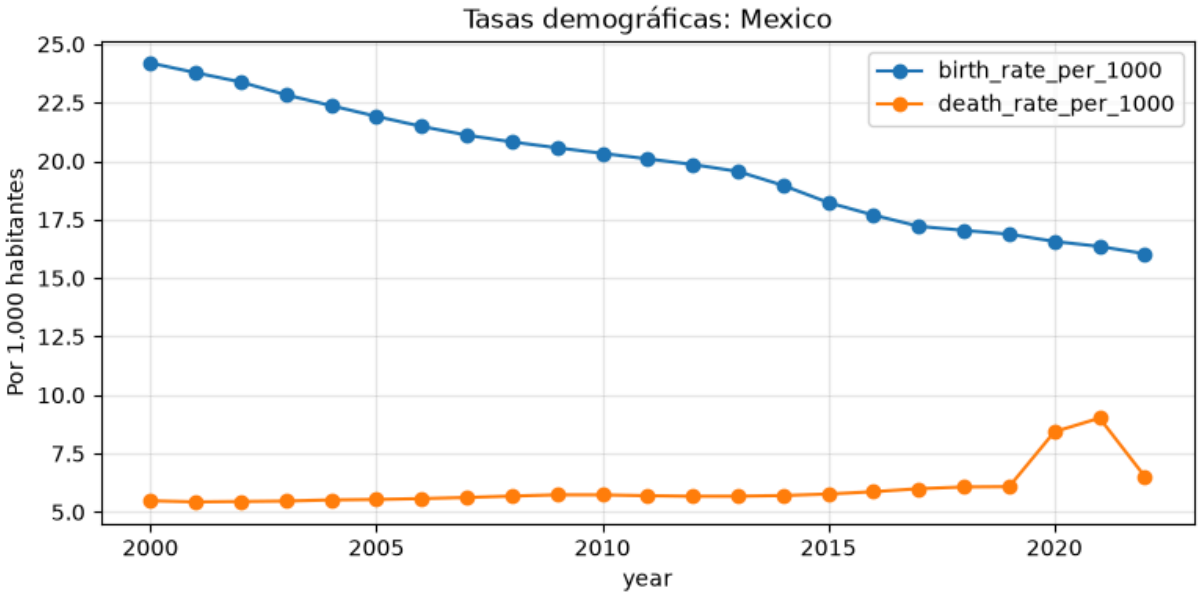
	entity	year	population	births	deaths	child_deaths_un
count	5818	5818.000000	5.818000e+03	5.818000e+03	5.818000e+03	5.818000
unique	253	NaN	NaN	NaN	NaN	
top	Afghanistan	NaN	NaN	NaN	NaN	
freq	23	NaN	NaN	NaN	NaN	
mean	NaN	2010.998109	1.813646e+08	3.694664e+06	1.412358e+06	1.996904
std	NaN	6.632822	8.095803e+08	1.664067e+07	6.115319e+06	9.607351
min	NaN	2000.000000	5.130000e+02	2.000000e+00	0.000000e+00	0.000000
25%	NaN	2005.000000	4.223308e+05	7.034000e+03	2.642000e+03	9.100000
50%	NaN	2011.000000	5.570254e+06	9.391100e+04	4.326450e+04	1.526000
75%	NaN	2017.000000	2.581040e+07	6.027868e+05	2.003098e+05	2.329175
max	NaN	2022.000000	8.021407e+09	1.460549e+08	6.972810e+07	1.018010

10. Visualizar una entidad disponible

Puedes cambiar `entidad` por cualquier nombre de `demografia["Entity"].unique()`.

In [16]:
```
entidad = "Mexico" if "Mexico" in demografia["entity"].values else demografia["enti
ejemplo = demografia[demografia["entity"] == entidad].sort_values("year")

ejemplo.plot(x="year", y=["birth_rate_per_1000", "death_rate_per_1000"],
             marker="o", figsize=(9, 4), title=f"Tasas demográficas: {entidad}")
plt.ylabel("Por 1,000 habitantes")
plt.grid(alpha=0.3)
plt.show()
```



11. Guardar el resultado

El CSV se guarda junto al notebook. Contiene solamente entidades y años compartidos por las cuatro fuentes.

In [17]:
```
from pathlib import Path

carpeta_salida = Path("../data/processed")
carpeta_salida.mkdir(parents=True, exist_ok=True)

salida = carpeta_salida / "demografia_owid_2000_2022.csv"
demografia.to_csv(salida, index=False)

print(f"Archivo guardado: {salida.resolve()} | filas: {len(demografia)}")
```

```
Archivo guardado: C:\Users\Santiago\Desktop\Escuela\5to Semestre ISSC\BigData\U3_1_modelos_ml\data\processed\demografia_owid_2000_2022.csv | filas: 5818
```